

PROGRAMMING II

COLLEZIONI

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.A. 2025/2026

COLLEZIONI JAVA



FRAMEWORK DELLE COLLEZIONI

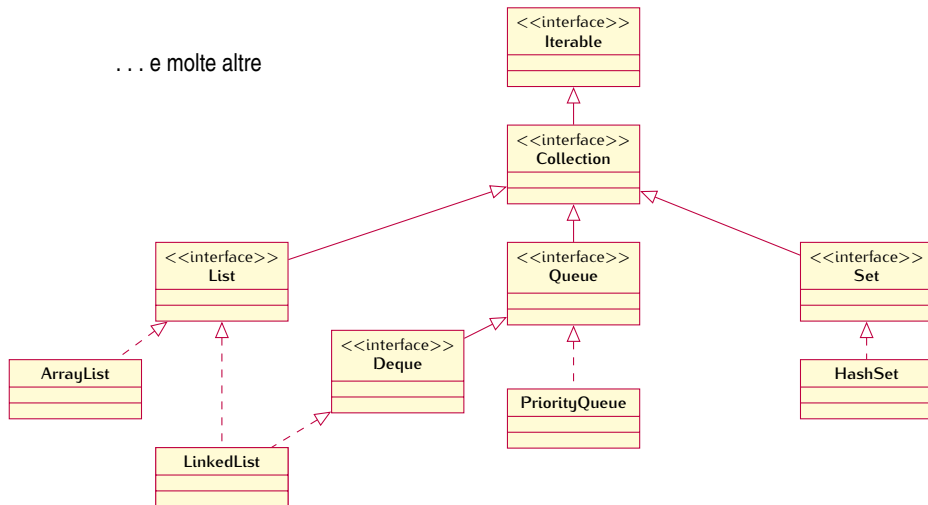
Insieme di classi e interfacce che implementano **collezioni** di elementi

- ▶ Insiemi, liste, code, . . .
- ▶ Diverse per come gli elementi sono raggruppati o acceduti
- ▶ Definite nel package `java.util`
- ▶ Tutte implementano l'interfaccia `Iterable<E>`

A partire da Java 5, il framework è stato ridefinito usando i generici

GERARCHIA DELLE COLLEZIONI

... e molte altre



DIVERSE IMPLEMENTAZIONI

Ogni collezione ha le proprie peculiarità

- ▶ Alcune collezioni sono ordinate e altre non lo sono
- ▶ Alcune collezioni hanno duplicati e altre no
- ▶ Alcune collezioni offrono più funzionalità e altre meno

Le implementazioni possono avere prestazioni diverse

		ArrayList	LinkedList
Recuperare un elemento	<code>get(n)</code>	costante	lineare
Aggiungere all'inizio	<code>add(0, e)</code>	lineare	costante

```
1 public interface Collection<E> extends Iterable<E> {
2     int size();
3     int hashCode();
4     boolean equals(Object obj); // equals non è generico ...
5     boolean isEmpty();
6     boolean contains(Object o);
7     boolean containsAll(Collection<?> c);
8     boolean add(E e);
9     boolean addAll(Collection<? extends E>);
10    boolean remove(Object o);
11    boolean removeAll(Collection<?> c);
12    void clear();
13    Object[] toArray();
14    Iterator<E> iterator(); // da Iterable<E>
15 }
```

CAMBIARE IMPLEMENTAZIONE

Da liste a insiemi, ad array, ... *// o a liste implementate diversamente*

```
Collection<Person> list = new LinkedList<>();           1
list.add(new Person("Joe"));                             2
list.add(new Person("Sam"));                             3
                                                         4
Collection<Person> set = new HashSet<>(); // o new HashSet<>(list); 5
set.addAll(list);                                         6
                                                         7
Object[] arrayO = set.toArray();                         8
Person[] arrayP = set.toArray(new Person[0]); // brutto ma utile 9
```

INTERFACCIA LIST

Le liste possono avere duplicati, e l'ordine di inserimento è preservato

- ▶ Gli elementi possono essere acceduti o inseriti a una data posizione

Metodi specifici dell'interfaccia `List<E>`

- ▶ Ottenere l'elemento alla posizione: `E get(int i);`
- ▶ Sostituire l'elemento alla posizione: `E set(int i, E e);`
- ▶ Aggiungere l'elemento alla posizione: `void set(int i, E e);`
- ▶ Rimuovere l'elemento alla posizione: `E remove(int i);`
- ▶ Posizione dell'elemento: `int indexOf(E e);`
- ▶ Elementi da l (incluso) a r (escluso): `List<E> subList(int l, int r);`

IMPLEMENTAZIONI DI LISTE

`ArrayList<E>` implementa `List<E>`

- ▶ Riservare spazio all'istanziamento: `ArrayList(int initialCapacity);`
- ▶ Ma il numero dei suoi elementi è ancora zero

`LinkedList<E>` implementa `List<E>`, `Queue<E>`

- ▶ Aggiungere in testa: `void addFirst(E e);`
- ▶ Ottenere l'ultimo: `E addLast();`
- ▶ Rimuovere il primo: `E removeFirst();`
- ▶ Inserire un elemento nello stack rappresentato dalla lista: `void push(E e);`
- ▶ Estrarre un elemento dallo stack rappresentato dalla lista: `E pop();`

INTERFACCIA SET

Gli insiemi non ammettono duplicati, e l'ordine di inserimento non è preservato

Non è possibile accedere o inserire elementi a una data posizione



L'interfaccia `Set<E>` non contiene metodi aggiuntivi rispetto a `Collection<E>`

Gli elementi nel suo `Iterator` vengono attraversati senza un ordine particolare



`Iterator`

IMPLEMENTAZIONE DI INSIEMI

Un'implementazione diffusa è `HashSet<E>`

- ▶ Costruita su una struttura dati chiave (hash) - valore
- ▶ Accesso diretto agli elementi basato su hash: tempo costante
- ▶ Calcolo dell'hash: tempo costante

`HashSet<E>` implementa `Set<E>`

- ▶ Mantiene internamente una `HashMap<E, Object>`
- ▶ Infatti, l'univocità degli oggetti si basa principalmente sul loro codice hash

HASHING

La classe `Object` contiene il metodo `public int hashCode();`

Restituisce un codice hash dell'oggetto

- ▶ Usato nelle strutture dati basate su hash (es. `HashSet`, `HashMap`)
- ▶ `HashSet`

È correlato con il metodo `public boolean equals(Object obj)`

`o1.equals(o2)` deve implicare `o1.hashCode() == o2.hashCode()`

Non è strettamente necessario il viceversa *// ma migliora le prestazioni*

`!o1.equals(o2)` può implicare `o1.hashCode() != o2.hashCode()`

HASHING (CONT.)

Regola aurea:

“se modifichi `equals()` devi modificare anche `hashCode()`, e viceversa”

Inoltre

Oggetti uguali devono essere associati allo stesso codice hash



Codici hash diversi devono essere associati a oggetti non uguali



CODICE HASH PERSONALIZZATO

```
class Student extends Person { 1
    int matricola; 2
    ... 3
    @Override 4
    public int hashCode() { 5
        return name.hashCode() * matricola; 6
    } 7
    // il metodo equals() deve essere coerente -- 8
    // deve verificare l'uguaglianza di name e matricola 9
} 10
```

A partire da Java 7, si può usare il metodo della classe `java.util.Objects`

`Objects.hash(name, matricola)`

IMPLEMENTAZIONE DI INSIEMI ORDINATI

Un'altra implementazione è `TreeSet<E>`

- ▶ Costruita su una struttura dati ad albero red-black // *albero binario auto-bilanciante*
- ▶ Gli elementi sono ordinati, ma non sono ammessi duplicati
- ▶ Computazionalmente costoso

Metodi aggiuntivi forniti

- ▶ Ottenere il primo elemento dell'insieme: `E first();`
- ▶ Ottenere l'ultimo elemento dell'insieme: `E last();`
- ▶ Elementi compresi tra from (incluso) e to (escluso):

```
TreeSet<E> subSet(E from, E to);
```

IMPLEMENTAZIONE DI INSIEMI ORDINATI (CONT.)

Differenza nelle prestazioni

- ▶ `HashSet` ha tempi costanti di inserimento, rimozione, ricerca
- ▶ `TreeSet` ha tempi logaritmici di inserimento, rimozione, ricerca

L'ordinamento e l'attraversamento avvengono in base a

- ▶ `TreeSet()`: ordinamento dagli elementi `Comparable<T>`
- ▶ `TreeSet(Comparator<E> comp)`: ordinamento dal `Comparator<T>` fornito

CONTENITORI ASSOCIATIVI

Una mappa è un insieme di associazioni tra chiavi e valori

(chiave) PassportId $\implies$ Person *(valore)*

Le chiavi devono essere univoche, e a ciascuna chiave può essere associato un solo valore

- ▶ In Java, chiavi e valori devono essere oggetti
- ▶ Implementazione dell'interfaccia `java.util.Map`
- ▶ Anche se parte del framework delle collezioni, non è una sottoclasse di `Collection`

MAPPE

```
public interface Map<K, V>
```

Metodi forniti dall'interfaccia

- ▶ Inserire un'associazione: `V put(K key, V values)`
- ▶ Ottenere il valore associato a una chiave: `V get(Object key)`
- ▶ Rimuovere l'associazione con una chiave specifica: `V remove(Object key)`
- ▶ Verificare se la mappa contiene una chiave: `boolean containsKey(Object key)`
- ▶ Verificare se la mappa contiene un valore: `boolean containsValue(Object value)`
- ▶ Restituisce l'insieme delle chiavi nella mappa: `Set<K> keySet()`
- ▶ Restituisce la collezione dei valori nella mappa: `Collection<V> values()`

IMPLEMENTAZIONI DI MAPPE

L'implementazione più comune è `HashMap` // *put e get in tempo costante*

```
Map<String, Person> people = new HashMap<>();           1
                                                         2
people.put("J4DF67", new Person("Sam"));                3
people.put("A3GH41", new Person("Joe"));                4
                                                         5
System.out.println(people.size()); // l'output è '2'    6
                                                         7
Person joe = people.get("A3GH41");                      8
if (joe == null) System.out.println("Not found");        9
                                                         10
if (!people.containsKey("A3GH41")) System.out.println("Not found"); 11
                                                         12
System.out.println(people.get("J4DF67")); // l'output è 'Sam' 13
```

LAVORARE CON LE COLLEZIONI



ITERATORI

Un'operazione comune è iterare sugli elementi di una collezione

- ▶ Ogni collezione è una sottoclasse di `Iterable<E>`
- ▶ Quindi ogni collezione restituisce un `Iterator<E>`
- ▶ Un `Iterator<E>` tiene traccia dell'ultimo elemento visitato e prosegue

L'interfaccia `Iterator<E>` fornisce tre metodi

- ▶ Verificare se è disponibile un altro elemento: `boolean hasNext();`
- ▶ Restituire l'elemento successivo e prosegue: `E next();`
- ▶ Rimuovere l'elemento corrente: `void remove();`

ITERATORI (CONT.)

Stampare tutti gli elementi di una lista

```
List<Integer> list = new ArrayList<>();           1
list.add(2);                                       2
list.add(3);                                       3

Iterator<Integer> it = list.iterator(); // usando un iteratore 4
while (it.hasNext()) {                             5
    System.out.println(it.next());                 6
}                                                    7
                                                    8
for (Integer i : list) { // usando il for-each      9
    System.out.println(i);                          10
}                                                    11
                                                    12
```

Perché dovremmo usare un iteratore?

ITERARE E MODIFICARE

Non è sicuro iterare una collezione modificandola allo stesso tempo

- A meno di non usare i metodi propri di `Iterator<E>`

```
int count = 0;                                     1
for (Integer i : list) {                           2
    System.out.println(i)                          3
    if (count == 2) list.remove(count); // errore a runtime -- 4
    count++;                                       // ConcurrentModificationException 5
}                                                  6
count = 0;                                         7
for (Iterator<Integer> it = list.iterator(); it.hasNext(); ) { 8
    System.out.println(it.next());                9
    if (count == 2) it.remove(); // ok            10
    count++;                                      11
}                                                  12
```

ORDINAMENTO

Le collezioni ordinabili (es. liste) hanno il metodo sovraccaricato `sort`

Su oggetti `Comparable<T>`

- ▶ `static <T extends Comparable<? super T>> void sort(List<T> list)`
- ▶ `Comparable` in fase di progettazione

Utilizzando un oggetto `Comparator<T>`

- ▶ `static <T> void sort(List<T> list, Comparator<? super T> cmp)`
- ▶ `Comparable` su richiesta

ORDINAMENTO CON COMPARABLE

```
class Person implements Comparable<Person> {  
    ...  
    @Override  
    public int compareTo(Person other) {  
        return this.name.compareTo(other.name);  
    }  
}
```

1
2
3
4
5
6
7
8

Per ordinare una lista `List<Person> list = new LinkedList<>()`

- ▶ Basta usare `Collections.sort(list);`
- ▶ Il metodo `compareTo()` viene usato per ordinare

ORDINAMENTO CON COMPARABLE (CONT.)

Perché <? super T> invece di <T>?

// <Person super Student>

```
// static <T extends Comparable<? super T>> void sort(List<T> list)      1
class Person implements Comparable<Person> {                             2
    ...                                                                    3
    @Override                                                              4
    public int compareTo(Person other) {                                    5
        return this.name.compareTo(other.name);                          6
    }                                                                        7
}                                                                            8
```

Abbiamo Student extends Person e vogliamo ereditare l'ordinamento di Person

- ▶ Qui Student estende (implicitamente) Comparable<Student>
- ▶ Ma Student non implementa Comparable<Person>

ORDINAMENTO CON COMPARATOR

Supponiamo che né Student né Person implementino Comparable

```
// static <T> void sort(List<T> list, Comparator<? super T> cmp)      1
class StudentComparator implements Comparator<Student> {              2
    public int compare(Student s1, Student s2) {                      3
        return s2.matr - s1.matr; // ordina in ordine inverso        4
    }                                                                    5
}                                                                        6
```

Per ordinare una lista `List<Student> list = new LinkedList<>()`

- ▶ Basta usare `Collections.sort(list, StudentComparator);`
- ▶ Definizione dinamica dell'ordinamento

ORDINAMENTO CON COMPARATOR (CONT.)

Supponiamo che né Student né Person implementino Comparable

```
Collections.sort(list, new Comparator<Student>() {           1
    public int compare(Student s1, Student s2) {           2
        return s2.matr - s1.matr;                           3
    }                                                         4
});                                                         5

Collections.sort(list, (s1, s2) -> { return s2.matr - s1.matr; }); 6
                                                         7
```

Naturalmente è possibile usare una classe anonima o una lambda

ORDINAMENTO DI ARRAY

Anche gli array hanno il metodo sovraccaricato `sort`

Su oggetti generici, ovvero su `Object`

- ▶ `static void sort(Object[] arr)`
- ▶ Confrontati secondo l'ordinamento naturale (viene usato `compareTo()`, se presente)

Utilizzando un oggetto `Comparator<T>`

- ▶ `static <T> void sort(T[] arr, Comparator<? super T> cmp)`
- ▶ Più type-safe

Ordinamento tramite `Arrays.sort(array);`

DOMANDE E RISPOSTE

